

Generics and Collections in Java 5

Maurice Naftalin
naftalin@morninglight.co.uk

Philip Wadler
wadler@inf.ed.ac.uk

This version is a draft. Please send comments to the addresses above.

Copyright ©2005 O'Reilly Media, Inc.

Part I

Introduction to Generics

Java 1.5 supports *generics*, the most significant change to the language since the addition of inner classes in Java 1.2 — some would say the most significant change to the language ever.

Say you wish to process lists. Some may be lists of bytes, others lists of strings, and yet others lists of lists of strings. In Java 1.4 this is simple. You can represent all three by the same class, called `List`, which has elements of class `Object`.

list of integers	<code>List</code>
list of strings	<code>List</code>
list of list of strings	<code>List</code>

In order to keep the language simple, you are forced to do some of the work yourself: you must keep track of the fact that you have a list of integers (or strings or lists of strings), and when you extract an element from the list you must cast it from class `Object` back to class `Integer` (or `Number` or `List`) before further processing. For instance, the Collections Framework in Java 1.4 makes extensive use of this idiom.

As Einstein said, everything should be as simple as possible but no simpler. And some might say the above is too simple. In Java 1.5 it is possible to represent lists a more direct way.

list of integers	<code>List<Integer></code>
list of string	<code>List<String></code>
list of list of strings	<code>List<List<String>></code>

Now the compiler keeps track of whether you have a list of integers (or numbers or lists of strings), and no explicit cast back to class `Integer` (or `Number` or `List<String>`) is required. In some ways, this is similar to *generics* in Ada or *templates* in C++, but the actual inspiration is *parametric polymorphism* as found in functional languages such as ML and Haskell.

Sun's work is influenced by a number of previous proposals, notably GJ by Bracha, Odersky, Stoutamire, and Wadler; an extension to GJ proposed by Igarashi and Viroli; and further development of this extension by Torgersen, Hansen, Ernst, von der Ahé, Bracha, and Gafter. Design of generics was carried out under the Java Community Process by a team led by Bracha, and including Odersky, Thorup, and Wadler (as parts of JSR 14 and JSR 201). Odersky's GJ compiler is the basis of Sun's current `javac` compiler.

Chapter 1

Getting started

Newton famously said “If I have seen farther than others, it is because I stand on the shoulders of giants”. The best programmers live by this motto, building on existing frameworks and reusable code where appropriate. The Java Collections Framework provides reusable interfaces and implementations for a number of common collection types, including lists, sets, queues, and maps. There is also a framework for comparing values, useful in sorting or building ordered trees. (Of course, not all programmers exploit reuse. As Hamming said of computer scientists, “Instead of standing on each others’ shoulders, we stand on each others’ toes.”)

Generics make it easier to use the Collections Framework, and the Collections Framework provides many examples of the use of generics; one might say that the two are made for each other. Here we will explain generics by examining their use in the context of the collections. A comprehensive introduction to collections appears later in this book, and a summary is printed in the endpapers. This section focusses on uses of collections that illustrate key features of generics.

Generics and collections work well with a number of other new features introduced in Java 1.5, including boxing and unboxing, functions that accept a variable number of arguments, and a new form of loop. We begin with an example that illustrates all of these. As we shall see, their combination is *synergistic*: the whole is greater than the sum of its parts.

Taking that as our motto, let’s do something simple with sums: put three numbers into a list and add them together. Here is how to do it in Java 1.5.

```
List<Integer> ints = Arrays.asList(1,2,3);
int sum = 0;
for (int n : ints) { sum += n; }
assert sum == 6;
```

You can probably read this code without much explanation, but let’s touch on the key new features. The interface `List` and the class `Arrays` are part of the Collections Framework (both are found in the package `java.util`). The type `List` is now *generic*, one write `List<E>` to indicate a list with elements of type `E`. Here we write `List<Integer>` to indicate that the elements of the list belong to the class `Integer`, the wrapper class that corresponds to the primitive type `int`. Boxing and unboxing operations, to convert from the primitive type to the wrapper class, are automatically inserted. The static method `asList` takes

any number of arguments, places them into an array, and returns a new list backed by the array. The new form of *foreach* loop is used to bind a variable successively to each element of the list, and the loop body adds these into the sum. The assertion statement, introduced in Java 1.4, is used to check that the sum is correct; when assertions are enabled, it throws an exception if the condition does not evaluate to true.

Here is how the same code looks in Java 1.4.

```
List ints = Arrays.asList( new Integer[] {
    new Integer(1), new Integer(2), new Integer(3)
} );
int sum = 0;
for (Iterator it = ints.iterator(); it.hasNext(); ) {
    int n = ((Integer)it.next()).intValue();
    sum += n;
}
assert sum == 6;
```

Reading this code is not quite so easy. Without generics, there is now no way to indicate in the type declaration what kind of elements are intended to be stored in the list, so instead of writing `List<Integer>` you write `List`. Now it is the coder rather than the compiler that is responsible for remembering the type of the list elements, so you must write the cast to `(Integer)` when extracting elements from the list. Without boxing and unboxing, you must explicitly allocate each object belonging to the wrapper class `Integer` and use the `intValue` method to extract the corresponding primitive `int`. Without functions that accept a variable number of arguments, you must explicitly allocate an array to pass to the `asList` method. Without the new form of loop, you must explicitly declare an iterator and advance it through the list.

By the way, here is how to do the same thing with an array in Java 1.4.

```
int[] ints = new int[] { 1,2,3 };
int sum = 0;
for (int i = 0; i < ints.size; i++) { sum += ints[i]; }
assert sum == 6;
```

This is slightly longer than the corresponding code that uses generics and collections, is arguably a bit less readable, and is certainly less flexible. Collections let you easily grow or shrink the size of the collection, or switch to a different representation when appropriate, such as a linked list or hash table or ordered tree. The introduction of generics, boxing and unboxing, *foreach* loops, and *varargs* in Java mark the first time where using collections is just as simple, perhaps even simpler, than using arrays.

Now let's look at each of these features in a little more detail.

1.1 Generics

An interface or class may be declared to take one or more type parameters, which are written in angle brackets, and should be supplied when you declare a variable belonging to the interface or class, or when you create a new instance of a class.

We saw one example above. Here is another.

```
List<String> words = new ArrayList<String>(2);
words.put(0, "hello");
words.put(1, "world");
String x = words.get(0);
String y = words.get(1);
System.out.println(x+" "+y);
```

In the Collections Framework, the class `ArrayList<E>` implements the interface `List<E>`. This trivial code snippet declares variable `words` to contain a list of string, creates an instance of a `ArrayList` of size 2, puts two strings into the list, and gets them out again.

In Java 1.4, the same code would be written as follows.

```
List words = new ArrayList(2);
words.put(0, "hello");
words.put(1, "world");
String x = (String)words.get(0);
String y = (String)words.get(1);
System.out.println(x+" "+y);
```

Without generics, the type parameters are omitted, but one must explicitly cast whenever an element is extracted from the list.

In fact, the byte code compiled from the two sources above will be identical. We say that generics are implemented by *erasure*, because the types `List<Integer>`, `List<String>`, and `List<List<String>>`, are at run-time all represented by the same type, which is equivalent to the non-generic type `List`.

Generics implicitly perform the same cast that is explicitly performed without generics. If such casts could fail, it might be hard to debug code written with generics. This is why it is reassuring that generics come with the following *cast-iron guarantee*: the implicit casts introduced by generics never fail. There is also some fine print on this guarantee: it applies only when no *unchecked warnings* have been issued. Later, we will discuss at some length what causes unchecked warnings to be issued, and explain the few situations where they are unavoidable.

Implementing generics by erasure has a number of important effects. It keeps things simple, in that generics do not add anything fundamentally new. It keeps things small, in that there is exactly one implementation of `List`, not one version for each type. And it eases evolution, since the same library can be accessed in both non-generic and generic form.

This last point is worth some elaboration. It means that one doesn't get nasty problems of maintaining two versions of the libraries, one that works with non-generic Java 1.4 code and another that works with generic Java 1.5 code. And it means that you can evolve your code to use generics one step at a time. At the byte-code level, code that doesn't use generics looks just like code that does. There is no need to switch to generics all at once, you can evolve your code by updating just one package, class, or method at a time to start using generics.

Another consequence of implementing generics by erasure is that array types differ in key ways from parameterized types. Executing

```
new String[2]
```

allocates an array of size two, and stores in the array an indication that its elements are all of type `String`. In contrast, executing

```
new ArrayList<String>(2)
```

allocates a list of size two, but does not store in the list any kind of indication of the type of its elements. Later, we will see how this leads to differences in subtyping and casting for parameterized types as compared to arrays.

Generics in Java resemble templates in C++. There are just two important things to bear in mind about the relationship between Java generics and C++ templates: syntax and semantics. The syntax is deliberately similar, the semantics is deliberately different.

Syntactically, angle brackets were chosen since they are familiar to C++ users. However, there is one difference in syntax. In C++, nested parameters require extra spaces, so you see things like this:

```
List< List<String> >.
```

This is because `>>` without the space denotes the right shift operator; there is a similar problem with `>>>`. In Java, no spaces are required, and it's fine to write this:

```
List<List<String>>.
```

This works through a minor complication to the Java grammar, which in this context treats the `>>` lexeme as closing two nested type parameters; similarly for `>>>`.

Semantically, Java generics are defined by *erasure*, whereas C++ templates are defined by *expansion*. In C++ templates, each instance of a template at a new type is compiled separately. If you use a list of integer, a list of string, and a list of list of string, then there will be three versions of the code. If you use lists of a hundred different types, there will be a hundred versions of the code, a problem known as *code bloat*. In Java, no matter how many types of list you use, there is always one version of the code, so no bloat.

Expansion may lead to more efficient implementation than erasure, since it offers more opportunities for optimization, particularly at primitive types such as `int`. For code manipulating large amounts of data – for instance, large arrays in scientific computing – this difference may be significant. However, in practice, for most purposes the difference in efficiency is not important, whereas the problems caused by code bloat can be significant.

1.2 Boxing and unboxing

Recall that every type in Java is either a *reference* type or a *primitive* type. A reference type is any class, instance, or array type. All reference types are subtypes of class `Object`, and any variable of reference type may be set to the value `null`. There are eight primitive types, and each of these has a corresponding library class, which is of reference type. The library classes are located in the package `java.lang`.

primitive	reference	primitive	reference
<code>byte</code>	<code>Byte</code>	<code>bool</code>	<code>Boolean</code>
<code>short</code>	<code>Short</code>	<code>char</code>	<code>Character</code>
<code>int</code>	<code>Integer</code>	<code>float</code>	<code>Float</code>
<code>long</code>	<code>Long</code>	<code>double</code>	<code>Double</code>

Conversion of a primitive type to the corresponding reference type is called *boxing* and of the reference type to the corresponding primitive type is called *unboxing*.

Java 1.5 automatically inserts boxing and unboxing coercions where appropriate. If an expression e_{int} of type `int` appears where a value of type `Integer` is expected, boxing converts it to the expression `new Integer( $e_{\text{int}}$ )`. (Actually, boxing is a bit trickier than this, see below.) If an expression e_{Integer} of type `Integer` appears where a value of type `int` is expected, unboxing converts it to the expression `$e_{\text{Integer}}$ .intValue()`.

Here again is the code to find the sum of a list of integers, conveniently packaged as a static method.

```
public static int sum (List<Integer> ints) {
    int s = 0;
    for (int n : ints) { s += n; }
    return s;
}
```

Why does the argument have type `List<Integer>` and not `List<int>`? Because type parameters must always be instantiated to reference types, not primitive types. Why does the result have type `int` and not `Integer`? Because result types may be either primitive or reference types, and it is more efficient to use the former than the latter. Unboxing occurs when each `Integer` in the list `ints` is bound to the variable `n` of type `int`.

Here we test two invocations of the method.

```
assert sum(Arrays.asList(1,2,3)) == 6;
assert sum(Arrays.asList(
    new Integer(1), new Integer(2), new Integer(3)
) == 6;
```

Boxing occurs when the `int` values 1, 2, 3 are passed to the method `asList`, so at run-time the first invocation behaves identically to the second.

We could rewrite the method, replacing each occurrence of `int` by `Integer`.

```
public static Integer sum_Integer (List<Integer> ints) {
    Integer s = 0;
    for (Integer n : ints) { s += n; }
    return s;
}
```

This code compiles, but does quite a lot of needless work. Each iteration of the loop unboxes the values in `s` and `i`, performs the addition, and boxes up the result again. Measurements show this routine is about 60% slower.

One subtlety of boxing and unboxing is that the equality operator `==` is defined differently on primitive and reference types. On type `int` it is equality of values, while on type `Integer` it is defined by object equality. So both of the following assertions succeed, using Sun's JVM for Java 1.5.

```
List<Integer> bigs = Arrays.asList(100,200,300);
assert sum_Integer(bigs) == sum(bigs);
assert sum_Integer(bigs) != sum_Integer(bigs);
```

In the first assertion, unboxing causes values to be compared, so the results are equal. In the second assertion, there is no unboxing, and the two method calls return distinct `Integer` objects, so the results are unequal — even though both `Integer` objects represent the same value, 600.

A further subtlety is that boxing an `int` value less than 128 is always guaranteed to return an object with not just the same value, but also the same identity. Hence, in contrast to the above, we have the following.

```
List<Integer> smalls = Arrays.asList(1,2,3);
assert sum_Integer(smalls) == sum(smalls);
assert sum_Integer(smalls) == sum_Integer(smalls);
```

This is because 6 is smaller than 128, so boxing the value 6 always returns exactly the same object. For values of 128 or greater it is not specified whether boxing the same value twice should return identical or distinct objects, so the inequality assertion above may either fail or succeed depending on the implementation.

1.3 Foreach

Here is a statement that computes the sum of a list of integers.

```
for (int n : ints) { s += n; }
```

This is called a *foreach* loop even though it is written with the keyword *for*. It is equivalent to the following.

```
for (Iterator<Integer> it = ints.iterator(); it.hasNext(); ) {
    int n = it.next();
    s += n;
}
```

This introduces the variable `it` of type `Iterator<Integer>` to iterate over the list `ints`. Note that unboxing occurs when the expression `it.next()` of type `Integer` is assigned to the variable `n` of type `int`.

The *foreach* loop can be applied to any object that implements the interface `Iterable<E>` (in package `java.lang`), which in turn refers to the interface `Iterator<E>` (in package `java.util`). These define the methods `iterator`, `hasNext` and `next`, used by the translation of the *foreach* loop. (Iterators also have a method `remove`, which is not used by the translation.)

```
interface Iterable<E> {
    public Iterator<E> iterator ();
}
interface Iterator<E> {
    public boolean hasNext ();
    public E next ();
    public void remove ();
}
```

All collections, sets, and lists in the Collections Framework implement the `Iterable<E>` interface; and libraries defined by other vendors or users may implement it as well.

The *foreach* loop may also be applied to an array.

```

public static int sum_array (int[] a) {
    int s = 0;
    for (int n : a) { s += n; }
    return s;
}

```

The foreach loop was deliberately kept simple, and only catches the most common case. One needs to explicitly introduce an iterator if one wishes to use the `remove` method or to iterate over more than one list in parallel. Here is a method that removes negative elements from a list of doubles.

```

public static void removeNegative(List<Double> v) {
    for (Iterator<Double> it = v.iterator(); it.hasNext();) {
        if (it.next() < 0) it.remove();
    }
}

```

Here is a method to compute the dot product of two vectors, represented as lists of doubles, both of the same length. (Given two vectors $u_1, \dots, u_n$ and $v_1, \dots, v_n$ it computes $u_1 * v_1 + \dots + u_n * v_n$.)

```

public static double dot(List<Double> u, List<Double> v) {
    double d = 0;
    Iterator<Double> u_it = u.iterator();
    Iterator<Double> v_it = v.iterator();
    while (u_it.hasNext()) {
        assert v_it.hasNext();
        d += u_it.next() * v_it.next();
    }
    assert !v_it.hasNext();
    return d;
}

```

Here the assertions about `v_it` are good practice, as they confirm the assumption that lists `u` and `v` are the same size.

1.4 Generic methods and varargs

Here is a method that accepts an array of any type and converts it to a list,

```

public static <E> List<E> asList (E[] arr) {
    List<E> list = new ArrayList<E>();
    for (E elt : arr) list.add(elt);
    return list;
}

```

The static method `asList` accepts an array of type `E[]` and returns a list of type `List<E>`. It does so for *any* type `E`, which is indicated by writing `<E>` at the beginning of the method signature. The scope of the type variable `E` is local to the method itself: it may appear in the method signature and the method body, but not outside the method. A method with a local type variable of this kind is called a *generic method*. The method may be invoked as follows.

```
List<Integer> ints = asList(new Integer[] { 1, 2, 3 });
List<String> words = asList(new String[] { "hello", "world" });
```

In the first line, boxing converts 1, 2, 3 from `int` to `Integer`.

Packing the arguments into an array is cumbersome. The *vararg* feature permits a special more convenient syntax for case when the last argument of a method is an array. To use this feature, we replace `E[]` by `E...` in the method declaration.

```
public static <E> List<E> asList (E... arr) {
    List<E> list = new ArrayList<E>();
    for (E elt : arr) list.add(elt);
    return list;
}
```

Now the method may be invoked as follows.

```
List<Integer> ints = asList(1, 2, 3);
List<String> words = asList("hello", "world");
```

This is just a shorthand for what we wrote above. At run-time, the arguments are packed into an array which is passed to the method, just as previously.

Any number of arguments may precede a last *vararg* argument. Here is a method that accepts a list and adds all the additional arguments to the end of the list.

```
public static <E> void addAll (List<E> list, E... arr) {
    for (E elt : arr) list.add(elt);
}
```

Whenever a *vararg* is declared, one may pass either a list of arguments to be implicitly packed into an array, or explicitly pass the array directly. Thus, the above method may be invoked as follows.

```
List<Integer> ints = new ArrayList<Integer>();
addAll(ints, 1, 2);
addAll(ints, new Integer[] { 3, 4 });
assert ints.toString().equals("[1, 2, 3, 4]");
```

We will later see that attempting to create an array containing a generic type must always issue an *unchecked* warning. Since *varargs* always create an array, they should be used only when the argument does not have a generic type. We return to this point later.

In the examples above, the type parameter to the generic method is inferred, but it may also be given explicitly. This may be particularly helpful when an argument is empty, since then the type information might not otherwise be inferred. The following initializes a list of integers to be the empty list.

```
List<Integer> ints = this.<Integer>asList();
```

Here the type parameter is required, because there is not enough information to infer the type of the right-hand side otherwise. In Java, the type of the left-hand side of an assignment is never used when determining the type of the right-hand side: the type of an expression depends only on the parts of the expression, not its context.

When a type parameter is passed to a generic method invocation it appears in angle brackets to the left, just as in the method declaration. The Java grammar requires that type parameters may only be in method invocations that use a dotted form. Here we have assumed the method `asList` is defined in the same class that invokes the code. We cannot shorten it as follows.

```
List<Integer> ints = <Integer>asList(); // compile-time error
```

The above is illegal because it would confuse the parser.

The Collection Framework defines static methods `java.util.Arrays.asList` and `java.util.Collection.addAll` similar to the ones above. The Collection Framework version of `asList` does not return an ordinary `ArrayList`, but instead a special form that is backed by a given array; and its version of `addAll` acts on general collections, not just lists.

1.5 Assertions

We clarify our code by liberal use of the *assert* statement. Each occurrence of *assert* is followed by a boolean expression that is expected to evaluate to `true`. If assertions are enabled and the expression evaluates to `false` then an `AssertionError` is thrown, including an indication of where the error occurred. Assertions are enabled by invoking the JVM with the `-ea` or `-enableAssertion` flag. Since assertions may not be enabled, an assertion should never have side effects upon which any non-assertion code depends.

1.6 Summing up

We have seen how generics, boxing and unboxing, *foreach* loops, and *varargs* work together to make Java code easier to write, illustrating this through the use of the Collections Framework.

Chapter 2

Subtyping and wildcards

Now that we've covered the basics, we can start to cover more advanced features of generics, such as subtyping and wildcards. In this section we'll review how subtyping works, and see how wildcards let you use subtyping in connection with generics. We'll illustrate our points with examples from the collections library.

2.1 Subtyping and The Substitution Principle

Subtyping is a key feature of object-oriented languages such as Java. In Java, one type is a *subtype* of another if it is related by an *extends* or *implements* clause. Here are some examples.

Integer is a subtype of Number
Double is a subtype of Number
Collection<E> is a subtype of Iterable<E>
List<E> is a subtype of Collection<E> ArrayList<E> is a subtype of Array<E>

Subtyping is transitive, meaning that if one type is a subtype of a second, and the second is a subtype of the third, then the first is a subtype of the third. So from the last two lines above it follows that `List<E>` is a subtype of `Collection<E>`. If one type is a subtype of another, we also say that the second is a *supertype* of the first. Every type is a subtype of `Object`, and `Object` is a supertype of every type. We also say, trivially, that every type is a subtype of itself.

The *Substitution Principle* tells us that wherever a value of one type is expected one may provide a value of any subtype of that type.

Substitution Principle. A variable of a given type may be assigned a value of any subtype of that type, and a method with a parameter of a given type may be invoked with an argument of any subtype of that type.

Consider the interface `Collection<E>`. One of its methods is `add`, which takes a parameter of type `E`.

```
interface Collections<E> {
    boolean add(E elt);
    ...
}
```

According to the Substitution Principle, if we have a collection of numbers, then we may add an integer or a double to it, because `Integer` and `Double` are subtypes of `Number`.

```
List<Number> nums = new ArrayList<Number>();
nums.add(2);
nums.add(3.14);
assert nums.toString().equals("[2, 3.14]");
```

Here, subtyping is used in two ways for each method call. The first call is permitted because `nums` has type `List<Number>`, which is a subtype of `Collection<Number>`, and `2` has type `Integer` (thanks to boxing), which is a subtype of `Number`. Similarly for the second call. In both calls, `E` is taken to be `Number`.

It may seem reasonable to expect that since `Integer` is a subtype of `Number`, it follows that `List<Integer>` is a subtype of `List<Number>`. But this is *not* the case, because the Substitution Principle would rapidly get us into trouble. It is not always safe to assign a value of type list of integer to a variable of type list of number. Consider the following code snippet.

```
List<Integer> ints = Arrays.asList(1,2);
List<Number> nums = ints; // compile-time error
nums.add(3.14);
assert ints.toString().equals("[1, 2, 3.14]"); // !!
```

The second line is a compile-time error, since `List<Integer>` is not a subtype of `List<Number>`. And it is a good thing that it is an error, since otherwise, as the fourth line shows, we could end up with a list of integers that contains a floating point number!

Arrays have a quite different behaviour, where `Integer[]` is a subtype of `Number[]`. We will compare the treatment of lists and arrays later (see Section 2.5).

Sometimes we would like lists to behave more like arrays, where we will accept not only a list with elements of a given type, but also a list with elements of any subtype of a given type. For this purpose we use *wildcards*.

2.2 Wildcards with extends

Another method in the `Collection` interface is `addAll`, which adds all of the members of one collection to another collection.

```
interface Collection<E> {
    ...
    boolean addAll(Collection<? extends E> c);
    ...
}
```

Clearly, given a collection of elements of type *E*, it is ok to add all members of another collection with elements of type *E*. What the quizzical phrase “*? extends E*” means is that it is also ok to add all members of a collection with elements of any type that is a *subtype* of *E*. The question mark is called a *wildcard*, since it stands for any type that is a subtype of *E*.

Here is an example. We create an empty list of numbers, and add to it first a list of integers and then a list of doubles.

```
List<Number> nums = new ArrayList<Number>();
List<Integer> ints = Arrays.asList(1, 2);
List<Double> dbls = Arrays.asList(2.78, 3.14);
nums.addAll(ints);
nums.addAll(dbls);
assert nums.toString().equals("[1, 2, 2.78, 3.14]");
```

The first call is permitted because *nums* has type *List<Number>*, which is a subtype of *Collection<Number>*, and *ints* has type *List<Integer>*, which is a subtype of *Collection<? extends Number>*. Similarly for the second call. In both calls, *E* is taken to be *Number*. If the method signature had been written without the wildcard then the calls would not have been permitted.

We can also use wildcards when declaring variables. Here is a variant of the example at the end of the preceding section, where we add a wildcard to the second line.

```
List<Integer> ints = Arrays.asList(1,2);
List<? extends Number> nums = ints;
nums.add(3.14); // ???
assert ints.toString().equals("[1, 2, 3.14]"); // !!!
```

Before, the second line raised a compile-time error (because *List<Integer>* is not a subtype of *List<Number>*), but the third line was fine (because one can add any number to a variable of type *List<Number>*). Now, the second line is fine (because *List<Integer>* is a subtype of *List<? extends Number>*), but the third line raises a compile time error. As before, the fourth line shows why one of the preceding lines had better be illegal!

In general, if a structure contains elements with a type of the form “*? extends E*”, then we can get elements out of the structure, but we cannot put elements into the structure. To put elements into the struture we need another kind of wildcard, as explained in the next section.

2.3 Wildcards with super

Here is a method that copies into a destination list all of the elements from a source list, from the convenience class *Collections*.

```
public static <T> void copy(List<? super T> dst,
                          List<? extends T> src) {
    for (int i = 0; i < src.length(); i++) {
        dst.set(i, src.get(i));
    }
}
```

What the quizzical phrase “? **super** T” means is that the destination list may have elements of any type that is a *supertype* of T, just as the source list may have elements of any type that is a *subtype* of T.

Here is an example, where we copy a list of integers into a list of objects.

```
List<Object> objs = Arrays.<Object>asList(2, 3.14, "four");
List<Integer> ints = Arrays.asList(5, 6);
Collections.<Number>copy(objs, ints);
assert objs.toString().equals("[5, 6, four]");
```

The call is permitted because `objs` has type `List<Object>` which is a subtype of `List<? super Number>` and `ints` has type `List<Integer>` which is a subtype of `List<? extends Number>`. Here T is taken to be `Number`, as indicated in the call. If the method signature had been written without the wildcards then the call would not have been permitted.

We have a choice to write the call several ways, all of which type check and all of which have the same effect.

```
Collections.<Object>copy(objs, ints); // (a)
Collections.<Number>copy(objs, ints); // (b)
Collections.<Integer>copy(objs, ints); // (c)
Collections.copy(objs, ints); // (d)
```

The last call leaves the type parameter implicit; it is taken to be `Integer`, since that is the smallest choice that works.

We could have omitted one of the two wildcards, at the cost of some flexibility. If we omitted the first wildcard then only calls (a) and (d) would work, with the type parameter taken to be `Object` in (d); and if we omitted the second wildcard then only calls (c) and (d) would work, with the type parameter taken to be `Integer` in (d). If we omitted *both* wildcards, then none of the calls would work.

2.4 The Get and Put Principle

It is good practice to insert wildcards whenever possible. But how do you decide *which* to use? Where should you use **extends** and where should you use **super** and where is it inappropriate to use a wildcard at all?

Fortunately, there is a simple principle that determines which is appropriate.

The Get and Put Principle. Use an **extends** wildcard when you only *get* values out of a structure, use a **super** wildcard when you only *put* values into a structure, and don't use a wildcard when you *both* get and put.

For example, look again at this method.

```
public static <T> void copy(List<? super T> dest,
                          List<? extends T> src);
```

The method gets values out of the source, so it is declared with an **extends** wildcard, and puts values into the destination, so it is declared with a **super** wildcard.

Whenever you use an iterator, you get values out of a structure, so use an **extends** wildcard. Here is a method that takes a collection of numbers, converts each to a double, and sums them up.

```

public static double sum(Collection<? extends Number> nums) {
    double sum = 0;
    for (Number num : nums) sum += num.doubleValue();
    return sum;
}

```

Since this uses **extends**, all of the following calls are legal.

```

List<Integer> ints = Arrays.asList(1,2,3);
assert sum(ints) == 6.0;

List<Double> doubles = Arrays.asList(2.78,3.14);
assert sum(doubles) == 5.92;

List<Number> nums = Arrays.<Number>asList(1,2,2.78,3.14);
assert sum(nums) == 8.92;

```

The first two calls would not be legal if **extends** was not used.

Whenever you use the **add** method, you are putting values into a structure, so use a **super** wildcard. Here is a method that takes a collection of numbers and an integer **n**, and puts the first **n** integers, starting from zero, into the collection.

```

public static void count(Collection<? super Integer> ints,
                        int n) {
    for (int i = 0; i < n; i++) ints.add(i);
}

```

Since this uses **super**, all of the following calls are legal.

```

List<Integer> ints = new ArrayList<Integer>();
count(ints, 5);
assert ints.toString().equals("[0, 1, 2, 3, 4]");

List<Number> nums = new ArrayList<Number>();
count(nums, 5);  nums.add(5.0);
assert nums.toString().equals("[0, 1, 2, 3, 4, 5.0]");

List<Object> objs = new ArrayList<Object>();
count(objs, 5);  objs.add("five");
assert objs.toString().equals("[0, 1, 2, 3, 4, five]");

```

The last two calls would not be legal if **super** was not used.

Whenever you both put values into and get values out of the same structure, you should not use a wildcard.

```

public static double sumcount (Collection<Number> nums,
                              int n) {
    count(nums, n);
    return sum(nums);
}

```

The collection is passed both to `sum` and `count`, so its element type must both extend `Number` (as `sum` requires) and be super to `Integer` (as `count` requires). The only two classes that satisfy both these constraints are `Number` and `Integer`, and we have picked the first of these. Here is a sample call.

```
List<Number> nums = new ArrayList<Number>();
double sum = sumcount(nums);
assert sum == 6;
```

Since there is no wildcard, the argument must be a collection of numbers.

If you don't like having to choose between `Number` and `Integer`, it might occur to you that if only Java let you write a wildcard with both `extends` and `super`, then you would not need to choose. For instance, one could write the following.

```
public static double sumcount(Collection<? super Integer
                               extends Number> coll,
                               int n)

// not legal Java!
```

Then we could call `sumcount` on either a collection of integers or a collection of numbers. But Java *doesn't* permit this. The only reason for outlawing it is simplicity, and conceivably Java might support such notation in the future. But until then, if you need to both put and get, don't use wildcards.

The Get and Put Principle also works the other way around. If an `extend` wildcard is present, then pretty much all you will be able to do is get but not put values of that type; and if a `super` wildcard is present, then pretty much all you will be able to do is put but not get values of that type.

For example, consider the following snippet using a list declared with `extends`.

```
List<Integer> ints = Arrays.asList(1,2,3);
List<? extends Number> nums = ints;
double dbl = sum(nums); // ok
nums.add(3.14); // compile-time error
```

The call to `sum` is fine, because we are getting values from the list, but the call to `add` is not, because we are putting a value into the list. This is just as well, since otherwise we could add a double to a list of integers!

Conversely, consider the following snippet using a list declared with `super`.

```
List<Object> objs = Arrays.<Object>asList(1,"two");
List<? super Integer> ints = objs;
ints.add(3); // ok
double dbl = sum(ints); // compile-time error
```

Now the call to `add` is fine, because we are putting a value into the list, but the call to `sum` is not, because we are getting a value from the list. This is just as well, because the sum of a list containing a string makes no sense!

There is one exception to the rule that you cannot get anything out from a type declared with `super`. Since every type extends `Object`, you can get values out a list declared with `super` only so long as you make no assumption stronger than that they have type `Object`.

```
List<Object> objs = Arrays.<Object>asList(1,"two");
List<? super Integer> ints = objs;
ints.add(3);
String str = "";
for (Object obj : ints) str += obj.toString();
assert str.equals("1two3");
```

In general, when the element type of a collection is declared with **super**, looping over the collection to get out elements at type **Object** will work, but an attempt to get out elements at any other type will fail.

2.5 Arrays

It is instructive to compare the treatment of lists and arrays in Java, keeping in mind the Substitution Principle and the Put and Get Principle.

In Java, array subtyping is *covariant*, meaning that type **S[]** is considered to be a subtype of **T[]** whenever **S** is a subtype of **T**. Consider the following code snippet, which allocates an array of integers, assigns it to an array of numbers, and then attempts to assign a float into the array.

```
Integer[] ints = new Integer[] {1,2,3};
Number[] nums = ints;
nums[2] = 3.14; // run-time error
```

Something is wrong with this program, since it puts a float into an array of integers! Where is the problem? Since **Integer[]** is considered a subtype of **Number[]**, by the Substitution Principle the assignment on the second line must be legal. Instead, the problem is caught on the third line, and it is caught at run-time. When an array is allocated (as on the first line) it is tagged with a description of the type of its elements, and every time it is assigned (as on the third line) the run-time type of the array is checked to ensure it is compatible with the value assigned to it.

In contrast, the subtyping relation for generics is *invariant*, meaning that type **List<S>** is *not* considered to be a subtype of **List<T>** except in the trivial case where **S** and **T** are identical. Here is a code snippet analogous to the one above, with lists replacing arrays.

```
List<Integer> ints = Arrays.asList(1,2,3);
List<Number> nums = ints; // compile-time error
nums.put(2, 3.14);
```

Since **List<Integer>** is not considered to be a subtype of **List<Number>**, the problem is detected on the second line, not the third, and it is detected at compile-time, not run-time.

Wildcards reintroduce covariant subtyping for generics, in that type **List<S>** is considered to be a subtype of **List<? extends T>**, when **S** is a subtype of **T**. Here is a third variant of the snippet.

```
List<Integer> ints = Arrays.asList(1,2,3);
List<? extends Number> nums = ints;
nums.put(2, 3.14); // compile-time error
```

Now it is again the third line which is in error, due to the Put and Get Principle: one cannot put a value into a type declared with an **extends** wildcard. In contrast to arrays, the problem is detected at compile-time, not run-time.

Wildcards also introduce *contravariant* subtyping for generics, in that type `List<T>` is considered to be a subtype of `List<? super T>`, when `S` is a subtype of `T`. Arrays do not support contravariant subtyping. For instance, there is no way to rewrite the routine `sum` so that it has a parameter of type `Integer[]` and one can pass in a `Number[]`.

Detecting problems at compile-time rather than at run-time is a great advantage. A minor advantage is that it is more efficient. One does not need to carry around a description of the element type at run-time, and one does not need to check against this description every time an assignment is performed. The major advantage is that a common family of errors is detected by the compiler. This improves virtually all aspects of the program's life cycle: coding, debugging, testing, and maintenance are all made easier, quicker, and less expensive.

Catching errors at compile-time rather than run-time is one reason why it is generally a good design principle to use collection classes in preference to arrays. A choice that eases coding, debugging, testing, and maintenance is often the right choice!

Nonetheless, there are a few cases where arrays are to be preferred to collections. Since primitive types don't have subtypes, arrays of primitive type won't have the problem of run-time errors, and will be much more efficient since they don't involve boxing. And despite the run-time tests, even arrays of reference type may be more efficient than collection classes with the current generation of compilers, so you may want to use arrays in crucial inner loops. As always, you should measure performance to justify such a design, especially since future compilers may optimize collection classes specially. Finally, in some cases arrays may be preferable for reasons of compatibility.

To summarize, it is better to detect errors at compile-time rather than run-time, but Java arrays are forced to detect certain errors at run-time by the decision to make array subtyping covariant. Was this a good decision? Before the advent of generics, it was absolutely necessary. For instance, look at the following methods, which are used to sort any array or to copy from one array to another.

```
public static void sort (Object[] a);
public static void fill (Object[] a, Object val);
```

However, now that we have generics, this property is no longer necessary, as we could give the methods the following type.

```
public static <T> void sort (T[] a);
public static <T> void fill (T[] a, T val);
```

In some sense, covariant arrays are an artefact of the lack of generics in earlier versions of Java. Once you have generics, covariant arrays are probably the wrong design choice; the only reason them in Java is the vital issue of backward compatibility.

2.6 Wildcards vs. type parameters

The method `contains` checks whether a collection contains a given object, and its generalization `containsAll` checks whether a collections contains every element of another collection. This section presents two alternate approaches to adding generics to these methods, one using wildcards and one using type parameters.

Here are the types that the methods have in Java 1.5.

```
interface Collection<E> {
    ...
    public boolean contains (Object o);
    public boolean containsAll (Collection<?> c);
    ...
}
```

The first method does not use generics at all! The second method is our first sight of an important abbreviation, where `Collection<?>` stands for `Collection<? extends Object>`. Extending `Object` is one of the most common uses of wildcards, so it makes sense to provide a short form for writing it.

These methods let us test for membership and containment.

```
Object obj = "one";
List<Object> objs = Arrays.<Object>asList("one", 2, 3.14, 4);
List<Integer> ints = Arrays.asList(2, 4);
assert objs.contains(obj);
assert objs.containsAll(ints);
assert !ints.contains(obj);
assert !ints.containsAll(objs);
```

The given list of objects contains both the string "one" and the given list of integers, but the given list of integers does not contain the string "one" nor does it contain the given list of objects.

The tests `ints.contains(obj)` and `ints.containsAll(objs)` might seem silly. Of course a list of integers won't contain an arbitrary object, such as the string "one". But it is permitted because sometimes such tests might succeed.

```
Object obj = 1;
List<Object> objs = Arrays.<Object>asList(1, 3);
List<Integer> ints = Arrays.asList(1, 2, 3, 4);
assert ints.contains(obj);
assert ints.containsAll(objs);
```

In this case the object may be contained in the list of integers because it happens to be an integer, and the list of objects may be contained within the list of integers because every object in the list happens to be an integer.

One might reasonably have chosen an alternative design for collection, where one can only test for containment for subtypes of the element type.

```
interface MyCollection<E>    // alternative design
    ...
    public bool contains (E o);
    public bool containsAll (Collection<? extends E> c);
    ...
```

Say we have a class `MyList` that implements `MyCollection`. Now the tests are legal only one way around.

```
Object obj = "one";
MyList<Object> objs = MyList.<Object>asList("one", 2, 3.14, 4);
MyList<Integer> ints = MyList.asList(2, 4);
assert objs.contains(obj);
assert objs.containsAll(ints)
assert !ints.contains(obj);           // compile-time error
assert !ints.containsAll(objs);      // compile-time error
```

The last two tests are illegal, because the type declarations require that we can only test whether a list contains element of a subtype of that list. So we can check whether a list of objects contains a list of integers, but not the other way around.

It is a matter of taste which of the two styles is better. The first permits more tests, while the second captures more errors at compile time (while also ruling out some sensible tests). The designers of the Java libraries chose the first, more liberal alternative, because someone using the Collection Framework *before* generics might well have written a test like `ints.containsAll(objs)`, and they would like that test to remain valid *after* generics were added to Java. However, when designing a new generic library, like `MyCollection`, when backwards compatibility is less important, then the design that catches more errors at compile time might make more sense.

The exact same design choice applies to other methods that contain `Object` or `Collection<?>` in their signature, such as `remove`, `removeAll`, and `retainAll`.

Chapter 3

Comparison and bounds

Now that we have the basics, let's look at some more advanced uses of generics. This section describes the interfaces `Comparable<E>` and `Comparator<E>` that are used to support comparison on elements. These interfaces are useful, for instance, if you want to find the maximum element of a collection or sort a list. Along the way we will introduce *bounds* on type variables, an important feature of generics that is particularly useful in combination with the `Comparable<E>` interface.

3.1 Comparable

The interface `Comparable<T>` contains a single method that can be used to compare one object to another.

```
interface Comparable<T> {  
    int compareTo(T o);  
}
```

The `compareTo` method returns a value that is negative, zero, or positive depending upon whether the argument is less than, equal to, or greater than the given object. The order specified by this interface is called the *natural order* for the object.

Typically, elements in certain classes can be compared only with elements of the same class. For instance, `Integer` implements `Comparable<Integer>`.

```
Integer int0 = 0;  
Integer int1 = 1;  
assert int0.compareTo(int1) < 0;
```

The comparison returns a negative number, since `0` precedes `1` in numerical order. Similarly `String` implements `Comparable<String>`.

```
String str0 = "zero";  
String str1 = "one";  
assert str0.compareTo(str1) > 0;
```

This comparison returns a positive number, since "zero" follows "one" in alphabetic order.

The type parameter to the interface allows nonsensical comparisons to be caught at compile time.

```
Boolean bool0 = false;
Boolean bool1 = true;
assert bool0.compareTo(bool1) > 0; // compile-time error
```

Here the comparison is illegal, because class `Boolean` does not implement interface `Comparable`. Here is another example.

```
Integer int0 = 0;
String str1 = "one";
assert int0.compareTo(str1) > 0; // compile-time error
```

One can compare an integer with an integer or a string with a string, but attempting to compare an integer with a string signals a compile-time error.

That's all there is to it for generics and comparison. But since the `Comparable` interface is so important, in the rest of this section we review important properties of this interface that one ought to know.

We use standard idioms for comparison. To check whether x is less than y , we write

```
x.compareTo(y) < 0
```

(this is the equivalent of writing $x < y$ for integers). Similarly, to check whether x is less than or equal to y , we write

```
x.compareTo(y) <= 0
```

(this is the equivalent of writing $x \leq y$ for integers).

The contract for the `Comparable<T>` interface specifies three obvious and simple properties. First, equal objects should have compare as the same:

```
if x.equals(y) then x.compareTo(y) == 0.
```

The converse usually holds, but not always; we'll later see an example where using `compareTo` to relate two objects returns zero while using `equals` on the same two objects returns false (see the end of Section 3.3). Second, reversing the order of arguments should reverse the result:

```
x.compareTo(y) < 0 if and only if y.compareTo(x) > 0.
```

This generalizes the property for integers: $x < y$ if and only if $y > x$. Third, comparison should be transitive:

```
if x.compareTo(y) <= 0
and y.compareTo(z) <= 0
then x.compareTo(z) <= 0
```

This generalizes the property for integers: if $x \leq y$ and $y \leq z$ then $x \leq z$.

It's worth pointing out a subtlety in the definition of comparison. Here is the right way to compare two integers.

```

class Integer implements Comparable<Integer> {
    ...
    public int compare (Integer that) {
        return this.value < that.value ? -1 :
               this.value == that.value ? 0 : 1 ;
    }
    ...
}

```

The conditional expression returns -1, 0, or 1 depending on whether the receiver is less than, equal to, or greater than the argument. You might think the following code work instead, since the method is permitted to return any negative integer if the receiver is less than the argument.

```

class Integer implements Comparable<Integer> {
    ...
    public int compareTo (Integer that) {
        // bad implementation -- don't do it this way!
        return this.value - that.value;
    }
    ...
}

```

This may give the wrong answer when there is overflow. For instance, when comparing a large negative value to a large positive value, the difference may be more than the largest value that can be stored in an integer, `Integer.MAX_VALUE`.

3.2 Maximum of a collection

In this section, we show how to use the `Comparable<T>` interface to find the maximum element in a collection. We begin with a simplified version. The actual version found in the Java 1.5 Collection Framework has a type signature that is a bit more complicated, and later we will see why.

Here is code to find the maximum element in a non-empty collection, from the class `Collections`.

```

public static <T extends Comparable<T>>
T max (Collection<T> coll) {
    T candidate = coll.iterator().next();
    for (T elt : coll) {
        if (candidate.compareTo(elt) < 0) candidate = elt;
    }
    return candidate;
}

```

This sets a candidate to the first element in the collection, then compares the candidate with each element in the collection, setting the candidate to the element when the element is larger. We use `iterator().next()` rather than `get(0)` to get the first element, because `get(0)` is not defined on collections other than lists. The method raises a `NoSuchElementException` when the collection is empty.

The highlighted phrase in angle brackets before the method signature declares the type parameter `T`, and indicates that the method can be instantiated for any type `T` that implements the interface `Comparable<T>`. We say that the type parameter `T` is *bounded* by the type `Comparable<T>`. As with wildcards, bounds for type variables are always indicated by the keyword `extends`, even when the bound is an interface rather than a class, as is the case here. Unlike wildcards, type variables must always be bounded using `extends`, never `super`.

For instance, `T` may be chosen to be `Integer` (since `Integer` implements `Comparable<Integer>`) or `String` (since `String` implements `Comparable<String>`).

```
List<Integer> ints = Arrays.asList(0,1,2);
assert Collections.max(ints) == 4;

List<String> strs = Arrays.asList("zero","one","two");
assert Collections.max(strs).equals("zero");
```

But we may not choose `T` to be `Boolean` (since `Boolean` does not implement `Comparable` at all).

```
List<Boolean> bools = Arrays.asList(false,true);
assert Collections.max(bools) == true; // compile-time error
```

As expected, here the call to `max` is illegal.

Here's an efficiency tip. The above implementation used a *foreach* loop to increase brevity and clarity. If efficiency was a pressing concern, one might want to rewrite the method to use an explicit iterator, as follows.

```
public static <T extends Comparable<T>>
T max (Collection<T> coll) {
    Iterator<T> it = coll.iterator();
    T candidate = it.next();
    while (it.hasNext()) {
        T elt = it.next();
        if (candidate.compareTo(elt) < 0) candidate = elt;
    }
    return candidate;
}
```

This allocates an iterator once instead of twice, and performs one less comparison.

Signatures for methods should be as general as possible to maximize utility. If one can replace a type parameter by a wildcard then one should do so. we can improve the signature of `max` by replacing

```
public static <T extends Comparable<T>>
T max (Collection<T> coll)
```

with

```
public static <T extends Comparable<? super T>>
T max (Collection<? extends T> coll)
```

Following the Get and Put Principle, we use `extends` with `Collection` because we *get* values of type `T` from the collection, and we use `super` with `Comparable` because we *put* value of type `T` into

the `compareTo` method. We'll see an example that would not type check if the `super` clause above was omitted in the next section.

If you look at the signature of this method in the Java library, you will see something that looks even worse than the above.

```
public static <T extends Object & Comparable<? super T>>
    T max(Collection<? extends T> coll)
```

This is there for backward compatibility, as explained at the end of Section 3.5.

3.3 A fruity example

The `Comparable<T>` interface gives fine control over what can and cannot be compared. Say that we have a class `Fruit` with subclasses `Apple` and `Orange`. Depending on how we set things up, we may *prohibit* comparison of apples with oranges or we may *permit* such comparison.

Figure 3.1 prohibits comparison of apples with oranges.

```
class Fruit {...}
class Apple extends Fruit implements Comparable<Apple> {...}
class Orange extends Fruit implements Comparable<Orange> {...}
```

Each fruit has a name and a size, and two fruits are equal if they have the same name and the same size. Apples are compared by comparing their sizes, and so are oranges. Since `Apple` implements `Comparable<Apple>`, it is clear that one can compare apples with apples, but not with oranges. The test code builds three lists, one of apples, one of oranges, and one containing mixed fruits. We may find the maximum of the first two lists, but attempting to find the maximum of the mixed list signals an error at compile time.

Figure 3.2 permits comparison of apples with oranges.

```
class Fruit implements Comparable<Fruit> ...
class Apple extends Fruit ...
class Orange extends Fruit ...
```

As before, each fruit has a name and a size, and two fruits are equal if they have the same name and the same size. Now any two fruits are compared by ignoring their names and comparing their sizes. Since `Fruit` implements `Comparable<Fruit>`, any two fruits may be compared. Now the test code can find the maximum of all three lists, including the one that mixes apples with oranges.

Recall that at the end of the previous section we extended the type signature of `compareTo` to use verbs *super*.

```
public static <T extends Comparable<? super T>>
    T max (Collection<? extends T> coll)
```

The second example shows why this wildcard is needed. If we want to compare two oranges, we take `T` in the above to be `Orange`.

```
Orange extends Comparable<? super Orange>
```

And this is true because both of the following hold.

```
Orange extends Comparable<Fruit>    and    Fruit super Orange
```

Without the `super` wildcard, finding the maximum of a list of oranges would be illegal, even though finding the maximum of a list of fruits is ok.

Also, note that this is a case where equality and comparison differ, in that using `compareTo` to relate two objects may return zero, while using `equals` to compare the same two objects may return false. This will happen, for instance, if one compares an apple and an orange of the same size.

3.4 Comparator

Sometimes we want to compare objects that do not implement the `Comparable` interface, or to compare objects using a different order than the one specified by that interface. The order specified by the `Comparable` interface is called the *natural order*, so we might regard `Comparator` as providing, so to speak, an unnatural order.

We can specify additional orderings using the `Comparator` interface, which contains a single method.

```
interface Comparator<T>
    public int compare(T o1, T o2);
```

The `compare` method returns a value that is negative, zero, or positive depending upon whether the first object is less than, equal to, or greater than the second object — just like `compareTo`.

For instance, here is a comparator that compares strings such that the shorter string is considered smaller. Only if two strings have the same length are they compared using the natural (alphabetic) order.

```
Comparator<String> sizeOrder
    = new Comparator<String> () {
        public int compare (String s1, String s2) {
            return
                s1.length() < s2.length() ? -1 :
                s1.length() > s2.length() ? 1 :
                s1.compareTo(s2) ;
        }
    };
};
```

In the natural alphabetic ordering, "three" is smaller than "two", while in the size ordering it is greater.

```
assert "two".compareTo("three") > 0;
assert sizeOrder.compare("two", "three") < 0;
```

In the size ordering, "one" is smaller than "two", since they have the same length so alphabetic order is used.

```
assert "one".compareTo("two") < 0;
assert sizeOrder.compare("one", "two") < 0;
```

```
import java.util.*;
class Fruit {
    protected String name;
    protected int size;
    public Fruit (String name, int size) {
        this.name = name; this.size = size;
    }
    public boolean equals (Object o) {
        if (o instanceof Fruit) {
            Fruit that = (Fruit)o;
            return this.name==that.name && this.size==that.size;
        } else return false;
    }
    protected int compareTo (Fruit that) {
        return this.size < that.size ? - 1 :
            this.size == that.size ? 0 : 1 ;
    }
}
class Orange extends Fruit implements Comparable<Orange> {
    public Orange (int size) { super("Orange", size); }
    public int compareTo (Orange o) { return super.compareTo(o); }
}
class Apple extends Fruit implements Comparable<Apple> {
    public Apple (int size) { super("Apple", size); }
    public int compareTo (Apple a) { return super.compareTo(a); }
}
class Test {
    public static void main (String... args) {

        Apple a1 = new Apple(1);  Apple a2 = new Apple(2);
        Orange o3 = new Orange(3);  Orange o4 = new Orange(4);

        List<Apple> apples = Arrays.asList(a1,a2);
        assert Collections.max(apples).equals(a2);

        List<Orange> oranges = Arrays.asList(o3,o4);
        assert Collections.max(oranges).equals(o4);

        List<Fruit> mixed = Arrays.<Fruit>asList(a1,o3);
        assert Collections.max(mixed).equals(o3);
        // compile-time error
    }
}
```

Figure 3.1: Prohibiting comparison of apples with oranges

```
class Fruit implements Comparable<Fruit> {
    String name;
    int size;
    public Fruit(String name, int size) {
        this.name = name; this.size = size;
    }
    public boolean equals (Object o) {
        if (o instanceof Fruit) {
            Fruit that = (Fruit)o;
            return this.name==that.name && this.size==that.size;
        } else return false;
    }
    public int compareTo(Fruit that) {
        return this.size < that.size ? - 1 :
            this.size == that.size ? 0 : 1 ;
    }
}
class Apple extends Fruit {
    public Apple (int size) { super("Apple", size); }
}
class Orange extends Fruit {
    public Orange (int size) { super("Orange", size); }
}
class Test {
    public static void main (String... args) {

        Apple a1 = new Apple(1);  Apple a2 = new Apple(2);
        Orange o3 = new Orange(3);  Orange o4 = new Orange(4);

        List<Apple> apples = Arrays.asList(a1,a2);
        assert Collections.max(apples).equals(a2);

        List<Orange> oranges = Arrays.asList(o3,o4);
        assert Collections.max(oranges).equals(o4);

        List<Fruit> mixed = Arrays.<Fruit>asList(a1,o3);
        assert Collections.max(mixed).equals(o3);
        // ok
    }
}
```

Figure 3.2: Permitting comparison of apples with oranges

In the Java libraries, one always has a choice between `Comparable` and `Comparator`. For every generic method with a type variable bounded by `Comparable`, there is another generic method with an additional argument of type `Comparator`. For instance, corresponding to

```
public static <T extends Comparable<? super T>>
    T max (Collection<? extends T> coll)
```

we also have

```
public static <T> T max (Collection<? extends T> coll,
                       Comparator<? super T> cmp)
```

The code for the new version of `max` can be seen in Figure 3.3. Compared to the old version, the only change is that where before we wrote `candidate.compareTo(elt)` now we write `cmp.compare(candidate, elt)`. Here are examples to find the maximum of the list using the natural order and using a given comparator.

```
Collection<String> strings
    = Arrays.asList("zero", "one", "two", "three", "four");
assert max(strings).equals("zero");
assert max(strings, sizeOrder).equals("three");
```

It is easy to define a comparator that provides the natural order.

```
public static <T extends Comparable<? super T>>
    Comparator<T> naturalOrder()
{
    return new Comparator<T> {
        compare (T o1, T o2) { return o1.compareTo(o2); }
    }
}
```

Using this, it is easy to define the version of `max` that uses the natural order in terms of the version that uses a given comparator.

```
public static <T extends Comparable<? super T>>
    T max (Collection<T> coll)
{
    return max(coll, Comparators.<T>naturalOrder());
}
```

A type parameter must be explicitly supplied for the invocation of the generic method `naturalOrder`. Java follows a general principle: the type of an expression depends only on the types of its subexpressions, not on the type expected by the context in which it is used. Since the method has no parameters it has no subexpressions that can be used to determine its type, so the type must be given explicitly. The same reasoning applies in general, so an explicit type parameter must always be provided when a generic method has no arguments.

Figure [reffig:comparators](#) also gives two overloaded versions of the generic method `reverseOrder`. One takes no arguments and returns a comparator for the reverse of the natural order, and the other takes

a comparator and returns a new comparator with the order reversed. Similar methods are provided in `java.util.Collections`.

It is easy to define versions of `min` for the natural order and for a given comparator, in terms of the version of `max` that accepts a comparator using the `reverseOrder` methods.

```
Collection<String> strings
    = Arrays.asList("zero", "one", "two", "three", "four");
assert min(strings).equals("four");
assert min(strings, sizeOrder).equals("one");
```

There are actually two different ways that `min` over the natural order can be defined. Here is the body of the method from the figure, followed by the alternative body.

```
return max(coll, Comparators.<T>reverseOrder());
return min(coll, Comparators.<T>naturalOrder());
```

The first of these is more direct so it is slightly faster. Measurements show a speedup of about 6%.

The Collections Framework does provide two version each of `min` and `max`, with the signatures given here. However, if you examine the source code of the library, you will see that none of the four is defined in terms of any of the others, each is defined directly instead. The more direct version is longer and harder to maintain, but faster. Measurements show a speedup of around 30%. Whether such a speedup is worth the code duplication depends on the situation in which the code is used. Since the Java utilities might well be used in a critical inner loop, the designers of the library were right to prefer speed of execution over economy of expression. But this is not always the case. An improvement of 30% may sound like a lot, but it's insignificant unless the total time of the program is significant and the routine appears in a heavily used inner loop. Don't make your own code needlessly prolix just to eek out a small improvement.

3.5 Multiple bounds

We have seen many examples where a type variable or wildcard is bounded by a single class or interface. In rare situations, it may be desirable to have multiple bounds, and we show how to do so here.

To demonstrate, we will use three interfaces from the Java library. Interface `Readable` has a method `read` to read into a buffer from a source, `Appendable` has a method `append` to copy from a buffer into a target, and `Closeable` has a method `close` to close a source or target. Possible sources and targets include files, buffers, streams, and so on.

For maximum flexibility, we might want to write a copy method that takes any source implementing both `Readable` and `Closeable` and any target implementing both `Appendable` and `Closeable`.

```

class Comparators {
    public static <T> T max (Collection<T> coll, Comparator<T> cmp) {
        T candidate = coll.iterator().next();
        for (T elt : coll) {
            if (cmp.compare(candidate, elt) < 0) { candidate = elt; }
        }
        return candidate;
    }
    public static <T extends Comparable<? super T>>
        T max (Collection<T> coll) {
        return max(coll, Comparators.<T>naturalOrder());
    }
    public static <T> T min (Collection<T> coll, Comparator<T> cmp) {
        return max(coll, reverseOrder(cmp));
    }
    public static <T extends Comparable<? super T>>
        T min (Collection<T> coll) {
        return max(coll, Comparators.<T>reverseOrder());
    }
    public static <T extends Comparable<? super T>>
        Comparator<T> naturalOrder () {
        return new Comparator<T> () {
            public int compare (T o1, T o2) { return o1.compareTo(o2); }
        };
    }
    public static <T> Comparator<T> reverseOrder (final Comparator<T> cmp) {
        return new Comparator<T> () {
            public int compare (T o1, T o2) { return cmp.compare(o2,o1); }
        };
    }
    public static <T extends Comparable<? super T>>
        Comparator<T> reverseOrder () {
        return new Comparator<T> () {
            public int compare (T o1, T o2) { return o2.compareTo(o1); }
        };
    }
    public static void main (String... args) {
        Comparator<String> sizeOrder = new Comparator<String> () {
            public int compare (String s1, String s2) {
                return
                    s1.length() < s2.length() ? -1 :
                    s1.length() > s2.length() ? 1 :
                    s1.compareTo(s2) ;
            }
        };
        Collection<String> strings = Arrays.asList("from","aaa","to","zzz");
        assert max(strings).equals("zzz");
        assert min(strings).equals("aaa");
        assert max(strings,sizeOrder).equals("from");
        assert min(strings,sizeOrder).equals("to");
    }
}

```

Figure 3.3: Comparators

```

public static <S extends Readable & Closeable,
               T extends Appendable & Closeable>
    void copy (S src, T trg, int size)
        throws IOException
{
    CharBuffer buf = CharBuffer.allocate(size);
    int i = src.read(buf);
    while (i >= 0) {
        buf.flip();
        trg.append(buf);
        buf.clear();
        i = src.read(buf);
    }
    src.close();
    trg.close();
}

```

This method repeatedly reads from the source into a buffer and appends from the buffer into a target. When the source is empty, it closes both the source and target. The first line specifies that *S* ranges over any type that implements both *Readable* and *Closeable*, and that *T* is ranges over any type that implements *Appendable* and *Closeable*. When multiple bounds on a type variable appear, they are separated by ampersands. (One cannot use a comma, since that is already used to separate declarations of type variables.)

For example, this method may be called with two files as source and target, or with the same two files wrapped in buffers as source and target.

```

int size = 32;
FileReader r = new FileReader("file.in");
FileWriter w = new FileWriter("file.out");
copy(r,w,size);
BufferedReader br = new BufferedReader(new FileReader("file.in"));
BufferedWriter bw = new BufferedWriter(new FileWriter("file.out"));
copy(br,bw,size);

```

Other possible sources include *FilterReader* or *PipedReader* or *StringReader*, and other possible targets include *FilterWriter*, *PipedWriter*, and *PrintStream*. But one could not use *StringBuffer* as a target, since it implements *Appendable* but not *Closeable*.

If you are picky, you may have spotted that all classes that implement both *Readable* and *Closeable* are subclasses of *Reader*, and almost all classes that implement *Appendable* and *Closeable* are subclasses of *Writer*. So you might wonder why we don't simplify the method signature like this.

```

public static void copy (Reader src, Writer trg, int size)

```

This will indeed admit most of the same classes, but not quite all. For instance, *PrintStream* implements *Appendable* and *Closeable* but is not a subclass of *Writer*. Furthermore, you can't rule out the possibility that some client of your code might have their own custom class that, say, implements *Readable* and *Closeable* but is not a subclass of *Reader*.

When multiple bounds appear, it is the first bound that is used for erasure. We saw a use of this in the section on maximum, above.

```
public static <T extends Object & Comparable<? super T>>  
    T max(Collection<? extends T> coll)
```

Without the highlighted text the erased type signature for `max` would have `Comparable` as the return type, whereas in legacy libraries the return type is `Object`.

Chapter 4

Evolution not revolution

One motto underpinning the design of generics for Java is *evolution not revolution*. It must be possible to migrate a large, existing body of code to use generics gradually (evolution) without requiring a radical all-at-once change (revolution). An important consequence is that old code compiles against the new Java libraries, avoiding the unfortunate situation where half your code needs old libraries and half your code needs new libraries.

This section shows how to add generics by considering a small example, a library for stacks that extends the Collections Framework, together with an associated client. We begin with the legacy stack library and client (written for Java 1.4). We then present the corresponding generic library and client (written for Java 1.5), which provides an opportunity to see how to write a generic class. Our example code is small, so it is easy to update to generics all in one go, but in practice the library and client may evolve separately. This is aided by *raw types*, which are the legacy counterpart of parameterized types.

The parts of the program may evolve in either order. One may have a generic library with a legacy client; this is the common case for anyone that uses the Collections Framework in Java 1.5 with legacy code. Or one may have a legacy library with a generic client; this requires that one provide generic signatures for the library without rewriting the entire library. We see two ways to do this. One is to combine generic signatures with legacy bodies, the other is to combine generic stub files with legacy class files. The latter is particularly useful when only class files are available.

In practice, the library and client may involve many interfaces and classes, and there may not even be a clear distinction between library and client. But the same principles discussed here still apply, and may be used to evolve any part of a program independently of any other.

4.1 Legacy library with legacy client

We begin with a simple library of stacks and an associated client, as presented in Figure 4.1. This is *legacy* code, written for Java 1.4 and its version of the Collections Framework. Like the Collections Framework, we structure the library as an interface `Stack` (which extends `List`), an implementation class `ArrayStack` (which extends `ArrayList`), and a utility class `Stacks` providing operations on stacks (analogous to `Collections`). The interface `Stack` provides just three methods, `empty`, `push`,

and `pop`. The implementation class `ArrayStack` provides a single constructor with no arguments, implemented in terms of the superclass, and implements the methods `empty`, `push`, and `pop` using methods `size`, `add`, and `remove` inherited from the superclass. The utility class provides just one method, `reverse`, which repeatedly pops from one stack and pushes into another.

The client allocates a stack, pushes a few integers onto it, pops an integer off, and then reverses the remainder into a fresh stack. Since this is Java 1.4, integers must be explicitly boxed when passed to `push`, and explicitly unboxed when returned by `pop`. The `toString` method used to examine the stack is inherited by `ArrayStack` from its superclass `ArrayList`.

4.2 Generic library and generic client

Next we update the library and client to use generics, as presented in Figure 4.2. This is *generic* code, written for Java 1.5 and its version of the Collections Framework. The major change is that the interface takes a type parameter, becoming `Stack<E>` (extending `List<E>`), and so does the implementing class, becoming `ArrayStack<E>` (extending `ArrayList<E>`). The type `Object` in the signatures and bodies of `push` and `pop` is replaced by the type parameter `E`. As this is our first look at a definition of a parameterized interface and class, it is worth noting that a type parameter is added just as one might expect to the occurrence of `Stack<E>` and `ArrayStack<E>` in the first line of the declaration, but is *not* added to the constructor `ArrayList` as it is already available from the enclosing declaration. The utility class `Stacks` does not take a parameter, but its method `reverse` becomes a generic method with argument and result of type `Stack<E>`. The updates to the client are straightforward.

In short, the conversion process is straightforward, requiring adding a few type parameters and replacing occurrences of `Object` by the appropriate type parameter. All differences between the legacy and generic versions can be spotted by comparing the highlighted portions of the two figures. The implementation of generics is designed so that the two versions generate essentially equivalent class files. Some auxiliary information about the types may differ, but the actual byte codes to be executed will be identical. Hence, executing the legacy and generic versions yields the same results. The fact that legacy and generic source yield the identical class files eases the process of evolution, as we next discuss.

4.3 Generic library with legacy client

Now let's consider the case where the library is updated to generics while the client remains in its legacy version. This may occur because there is not time to convert everything all at once, or because the library and client are controlled by different organizations. This corresponds to the most important case of backward compatibility, where the generic Collections Framework of Java 1.5 must still work with legacy clients written against the Collections Framework in Java 1.4.

In order to support evolution, whenever a parameterized type is defined, Java also recognizes the corresponding unparameterized version of the type, called a *raw type*. For instance, the parameterized type `Stack<E>` corresponds to the raw type `Stack`, and the parameterized type `ArrayStack<E>` corresponds to the raw type `ArrayStack`. Java permits a raw type to be passed where the corresponding parameterized type is expected, or vice versa, but it generates an *unchecked* warning in such circumstances.

To be specific, consider compiling the generic source for `Stack<E>`, `ArrayStack<E>` and `Stacks` from Figure 4.2 (say, in directory `g`) with the legacy source for `Client` from Figure 4.1 (say, in

```

interface Stack extends List {
    public boolean empty();
    public void push(Object elt);
    public Object pop();
}

class ArrayStack extends ArrayList implements Stack {
    public ArrayStack() { super(); }
    public boolean empty() { return size()==0; }
    public void push(Object elt) { add(elt); }
    public Object pop() {
        Object elt = get(size()-1);
        remove(size()-1);
        return elt;
    }
}

class Stacks {
    public static Stack reverse(Stack in) {
        Stack out = new ArrayStack();
        while (!in.empty()) {
            Object elt = in.pop();
            out.push(elt);
        }
        return out;
    }
}

class Client {
    public static void main (String[] args) {
        Stack stack = new ArrayStack();
        for (int i = 0; i<4; i++) stack.push(new Integer(i));
        assert stack.toString().equals("[0, 1, 2, 3]");
        int top = ((Integer)stack.pop()).intValue();
        assert top==3 && stack.toString().equals("[0, 1, 2]");
        Stack reverse = Stacks.reverse(stack);
        assert stack.empty();
        assert reverse.toString().equals("[2, 1, 0]");
    }
}

```

Figure 4.1: Legacy library and legacy client

```

interface Stack<E> extends List<E> {
    public boolean empty();
    public void push(E elt);
    public E pop();
}
class ArrayStack<E> extends ArrayList<E> implements Stack<E> {
    public ArrayStack() { super(); }
    public boolean empty() { return size()==0; }
    public void push(E elt) { add(elt); }
    public E pop() {
        E elt = get(size()-1);
        remove(size()-1);
        return elt;
    }
}
class Stacks {
    public static <E> Stack<E> reverse(Stack<E> in) {
        Stack<E> out = new ArrayStack<E>();
        while (!in.empty()) {
            E elt = in.pop();
            out.push(elt);
        }
        return out;
    }
}
class Client {
    public static void main (String[] args) {
        Stack<Integer> stack = new ArrayStack<Integer>();
        for (int i = 0; i<4; i++) stack.push(i);
        assert stack.toString().equals("[0, 1, 2, 3]");
        int top = stack.pop();
        assert top==3 && stack.toString().equals("[0, 1, 2]");
        Stack<Integer> reverse = Stacks.reverse(stack);
        assert stack.empty();
        assert reverse.toString().equals("[2, 1, 0]");
    }
}

```

Figure 4.2: Generic library with generic client

directory `l`). Sun's Java 1.5 compiler yields the following message.

```
% javac g/Stack.java g/ArrayStack.java g/Stacks.java l/Client.java
Note: Stack.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
```

The unchecked warning indicates that the compiler cannot offer the same safety guarantees that are possible when generics are used uniformly throughout. However, when the generic code was generated by updating legacy code, we know that equivalent class files are produced from both, and hence (despite the unchecked warning) running a legacy client with the generic library will yield the same result as running the legacy client with the legacy library. Here we assume that the only change in updating the library was to introduce generics, and that no change to the behaviour was introduced, either on purpose or by mistake.

If we follow the suggestion above and re-run the compiler with the appropriate switch enabled, we get more details.

```
% javac -Xlint:unchecked g/Stack.java g/ArrayStack.java \
%   g/Stacks.java l/Client.java
l/Client.java:6: warning: [unchecked] unchecked call
to push(E) as a member of the raw type Stack
    for (int i = 0; i<4; i++) stack.push(new Integer(i));
                                   ^
l/Client.java:10: warning: [unchecked] unchecked conversion
found   : Stack
required: Stack<E>
    Stack reverse = Stacks.reverse(stack);
                                   ^
l/Client.java:10: warning: [unchecked] unchecked method invocation:
<E>reverse(Stack<E>) in Stacks is applied to (Stack)
    Stack reverse = Stacks.reverse(stack);
                                   ^
3 warnings
```

Not every use of a raw type gives rise to a warning. Putting a value into a raw type issues a warning (hence the warning for the invocation of `push`) but getting a value from a raw type is safe (hence no warning for the invocation of `pop`); this works for reasons similar to the Put and Get Principle. Passing a raw type where a parameterized type is expected issues a warning (hence the warning when passing an argument to `reverse`) but passing a parameterized type where a raw type is expected is safe (hence no warning for getting the result from `reverse`); this works for reasons similar to the Substitution Principle.

The most important case of using generic libraries with legacy clients is that of using the Java 1.5 Collections Framework with legacy code written for the Java 1.4 Collections Framework. In particular, applying the Java 1.5 compiler to the legacy code in Figure 4.1 also issues unchecked warnings, because of the uses of generic versions of `List` from the legacy version of `ArrayStack`. Here is what happens when we compile legacy versions of all the files.

```
% javac -Xlint:unchecked l/Stack.java l/ArrayStack.java \
%   l/Stacks.java l/Client.java
l/ArrayStack.java:5: warning: [unchecked] unchecked call to add(E)
as a member of the raw type ArrayStack
    public void push(Object elt)  add(elt);
                                ^
1 warning
```

Here the warning for the use of generic method `add` in legacy method `push` is issued for similar reasons to the previous warning for use of generic method `push` from the legacy client.

It is never a good idea to cry wolf. In the case of pure legacy code, such warnings can be turned off by using the `-source 1.4` switch.

```
% javac -source 1.4 l/Stack.java l/ArrayStack.java \
%   l/Stacks.java l/Client.java
```

This compiles the legacy code and issues no warnings or errors. This method of turning off warnings is only applicable to true legacy code, with no 1.5 features, generic or otherwise. One can also turn off unchecked warnings by using annotations, as described below, and this works even with 1.5 features.

4.4 Legacy library with generic client

Usually it makes sense to update the library before the client, but there may be cases when you wish to do it the other way around. For instance, you may be responsible for maintaining the client but not the library; or the library may be large, so you want to update it gradually rather than all-at-once; or you may have class files for the library, but no source.

In such cases, it makes sense to update the library to use parameterized types in its method signatures, but not to change the method bodies. There are two ways to do this. One involves making minimal alterations to the source, the other involves creating stub files.

4.5 Evolving a library — minimal changes

The minimal changes technique is shown in Figure 4.3. Here the source of the library has been edited, but only to change method signatures, not method bodies. The exact changes required are highlighted in boldface.

To be precise, the changes required are (a) adding type parameters to interface or class declarations as appropriate (for interface `Stack<E>` and class `ArrayStack<E>`), (b) adding type parameters to any newly parameterized interface or class in an extends or implements clause (for `Stack<E>` in the implements clause of `ArrayStack<E>`), (c) adding type parameters to each method signature as appropriate (for `push`, and `pop` in `Stack<E>` and `ArrayStack<E>`, and for `reverse` in `Stacks`), (d) adding a cast to any return statement, when the return type is a type parameter (for `pop` in `ArrayStack<E>`, where the return type is `E` — without this cast one will get an error rather than an unchecked warning), and (e) optionally adding annotations to suppress unchecked warnings (for classes `ArrayStack<E>` and `Stacks`).

It is worth noting a few changes that we did *not* need to make. In method bodies, we can leave occurrences of `Object` as they stand, and we do not need to add type parameters to any occurrences of raw types (see the first line of `pop` in `ArrayStack`, and of `reverse` in `Stacks`). Also, we only need to add a cast to a return clause when the return type is a type parameter (as in `pop`) but not when the return type is a parameterized type (as in `reverse`). Finally, we didn't need to parameterize occurrences of other library classes (such as the occurrences of `List` in the extends clause of `Stack<E>` or of `ArrayList` in the extends clause of `ArrayStack<E>`). These changes are not required because the types involved are raw types, and so already yield unchecked warnings rather than errors. We might want to make them anyway, but it is helpful to know that they are not required when a large body of code needs to be changed and we want to keep the work to a minimum.

With these changes, the library will compile successfully, although it will issue a number of unchecked warnings. To indicate that we expect unchecked warnings when compiling the library classes, these have been annotated to suppress such warnings.

```
@SuppressWarnings("unchecked");
```

This prevents the compiler from crying wolf — we've told it not to issue unchecked warnings that we expect, so it will be easy to spot any that we *don't* expect. In particular, once we've updated the library, we should not see any unchecked warnings from the client. Note well that we've suppressed warnings on the library classes, but *not* on the client! (The suppress warnings annotation does not work in the first version of Java 1.5.)

There is no way to get rid of the unchecked warnings generated by compiling the library, short of updating the entire library source to use generics. This is entirely reasonable, as unless the entire source is updated there is no way the compiler can check that the declared generic types are correct. Indeed, unchecked warnings are warnings — rather than errors — largely because they support the use of this technique. It is best to use this technique only if you are sure that the generic signatures are in fact correct. The best practice is to use this technique only as an intermediate step in evolving code to use generics throughout.

4.6 Evolving a library — stubs

The stubs technique is shown in Figure 4.4. Here we write stubs with generic signatures but no bodies. We compile the generic client against the generic signatures, but run the code against the legacy class files. This technique is appropriate in when the source is not released, or when others are responsible for maintaining the source.

To be precise, we introduce the same modifications to interface and class declarations and method signatures as with the minimal changes technique, except we completely delete all executable code, replacing each method body by code that throws a `StubException` (a new exception that extends `UnsupportedOperationException`).

When we compile the generic client, we do so against the class files generated from the stub code, which contain appropriate generic signatures (say, in directory `g`). When we run the client, we do so against the original legacy class files (say, in directory `l`).

```
% javac -classpath g g/Client.java
% java -ea -classpath l g/Client
```

```

interface Stack<E> extends List {
    public boolean empty();
    public void push(E elt);
    public E pop();
}

@SuppressWarnings("unchecked")
class ArrayStack<E> extends ArrayList implements Stack<E> {
    public boolean empty() { return size()==0; }
    public void push(E elt) { add(elt); }
    public E pop() {
        Object elt = get(size()-1);
        remove(size()-1);
        return (E)elt;
    }
}

@SuppressWarnings("unchecked")
class Stacks {
    public static <E> Stack<E> reverse(Stack<E> in) {
        Stack out = new ArrayStack();
        while (!in.empty()) {
            Object elt = in.pop();
            out.push(elt);
        }
        return out;
    }
}

class Client {
    public static void main (String... args) {
        Stack<Integer> stack = new ArrayStack<Integer>();
        for (int i = 0; i<4; i++) stack.push(i);
        assert stack.toString().equals("[0, 1, 2, 3]");
        int top = stack.pop();
        assert top==3 && stack.toString().equals("[0, 1, 2]");
        Stack<Integer> reverse = Stacks.reverse(stack);
        assert stack.empty();
        assert reverse.toString().equals("[2, 1, 0]");
    }
}

```

Figure 4.3: Evolving a library — minimal changes

```
interface Stack<E> extends List {
    public boolean empty();
    public void push(E elt);
    public E pop();
}

class StubException extends UnsupportedOperationException {}

class ArrayStack<E> extends ArrayList implements Stack<E> {
    public boolean empty() { throw new StubException(); }
    public void push(E elt) { throw new StubException(); }
    public E pop() { throw new StubException(); }
}

class Stacks {
    public static <E> Stack<E> reverse(Stack<E> in) {
        throw new StubException();
    }
}
```

Figure 4.4: Evolving a library — stubs

Again, this works because the class files generated for legacy and generic files are essentially identical, save for auxiliary information about the types. In particular, the generic signatures that client is compiled against match the legacy signatures (apart from auxiliary information about type parameters), so the code runs successfully and gives the same answer as previously.

4.7 Conclusions

To review, we have seen both generic and legacy versions of a library and client. These generate equivalent class files, which greatly eases evolution. One can use a generic library with a legacy client, or a legacy library with a generic client. In the latter case, one can update the legacy library with generic method signatures, either by minimal changes to the source or by use of stub files.

The secret ingredient which makes all of this work is the designers' decision that generic code should generate essentially the same class files as legacy code. This explains why generic types do not carry information about type parameters at run time, in contrast to the design of classes which do carry information about array element types at run time.

It is interesting to compare the design of generics in Java and in C#. In C#, generic types do carry information about type information at run time. In some ways, this increases the power of the language. Consider casting from `Object` to `List<String>`. In Java this raises an unchecked warning, because at run-time the only way to tell one has a list of string is to check that every element of the list is string. In C#

the object is labeled with its type, so the equivalent cast can be easily checked at run time. However, evolution from legacy to generic code in C# is more difficult. In particular, adaptor classes are needed to view a legacy class as a generic class or vice versa. If one had a nested type, such as `List<List<String>>`, then complex nesting of adaptors will be required. In contrast, as we've seen, evolution in Java can be quite straightforward.